eman ta zabal zazu
Universidad del País Vasco
Euskal Herriko Unibertsitatea
INFORMATIKA FAKULTATEA
FACULTAD DE INFORMÁTICA

PROIEKTUAREN DISEINU PATROIAK

Oier Amilibia, Pablo Carmona eta Joritz Arocena
Software Ingenieritza II
26-27 ikasturtea

Aurkibidea

Aurkibidea **2**
Egileak eta githubeko esteka **2**
1. Factory method **3**
1.1 Lortutako UML 3
1.2 Egindako kode aldaketak 3
2. Iterator Patroia **5**
2.1 Lortutako UMLa 5
2.2 Egindako kode aldaketak 5
2.3 Lortutako emaitzak 7
3. Adapter Patroia **8**
3.1 Lortutako UML 8
3.2 Egindako kode aldaketak 8
3.3 Lortutako emaitzak 9

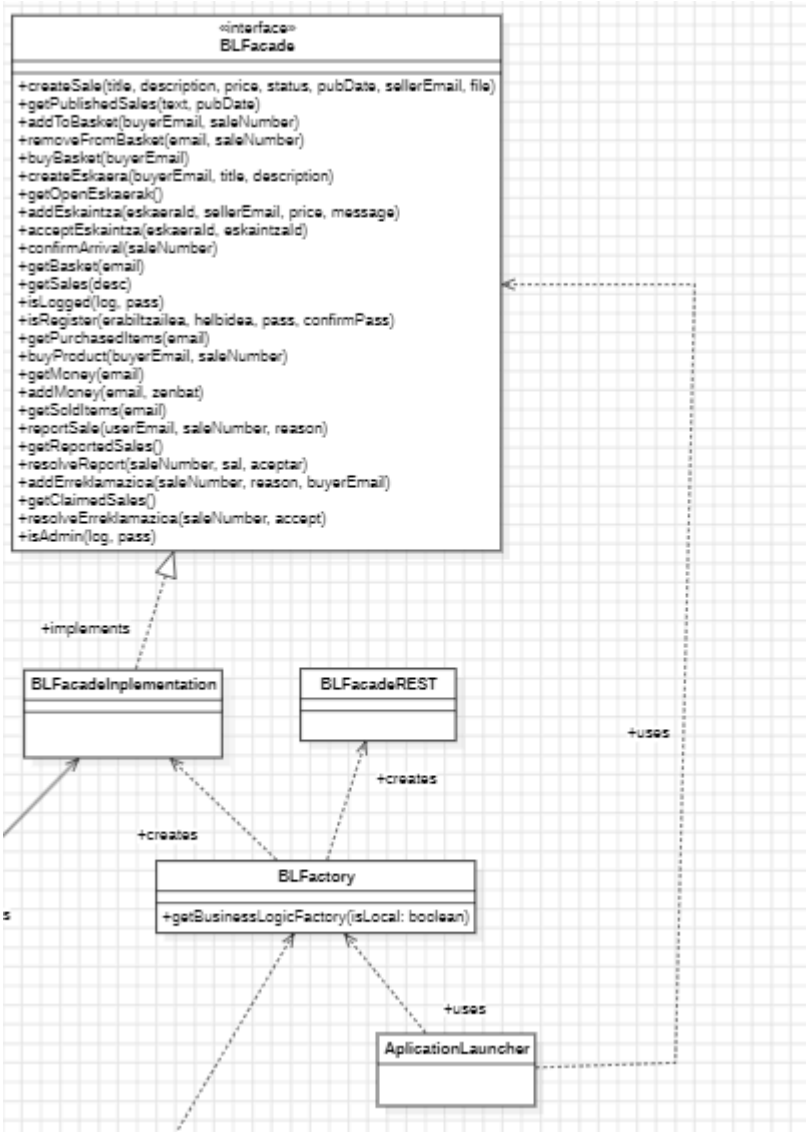
Egileak eta githubeko esteka

Github:https://github.com/jarocena013/Market726-is2-si2

Egileak: Oier Amilibia, Pablo Carmona eta Joritz Arocena

1. Factory method

1.1 Lortutako UML



Lortu den UML honekin lortzen dena da, aplicationLauncher klaseak BLFacade bat sortu nahi duen bakoitzean BLFactory klaseari egingo diola dei, eta honek erabakiko du zein motatako Facade nahi duen sortu, eta sortu eta ondoren bere balioa itzuliz.

1.2 Egindako kode aldaketak

Egin den aldaketa nagusia BusinessLogic-a sortzeko factory klase bat izan da. Klase honek parametro gisa jasotzen du ea modu lokaleana sortu behar duen edo kanpo zerbitzu baten bdiez sortu behar duen. Jasotzen duen parametroaren arabera bata edo bestea sortuko du, eta sortutakoa itzultzen du. Honako kodea izango du klase horrek:

```
public class BLFactory {

    public BLFacade getBusinessLogicFactory(boolean isLocal) {
        ConfigXML c = ConfigXML.getInstance();
        BLFacade appFacadeInterface = null;

        try {
            if (isLocal) {
                // ConcreteProduct 1: Local
                DataAccess da = new DataAccess();
                appFacadeInterface = new BLFacadeImplementation(da);
            } else {
                // ConcreteProduct 2: Remoto (Web Service)
                String serviceName = "http://" + c.getBusinessLogicNode() + ":" + c.getBusinessLogicPort() + "/ws/"
                        + c.getBusinessLogicName() + "?wsdl";
                URL url = new URL(serviceName);

                QName qname = new QName("http://businessLogic/", "BLFacadeImplementationService");
                Service service = Service.create(url, qname);

                appFacadeInterface = service.getPort(BLFacade.class);
            }
        } catch (Exception e) {
            System.out.println("Error in BLFactory: " + e.toString());
        }

        return appFacadeInterface;
    }
}
```

Behin factory klasea sortuta dugula, hurrengo kode aldaketa AplicationLauncher klasean egin behar da. Klase honek orain BLFacade bat sortu nahi duenean faktoriari deituko dio eta honek sortuko du BLFacadea. Honelako kodea gelditu da AplicationLauncher klasean:

```
public class ApplicationLauncher {

    public static void main(String[] args) {

        ConfigXML c = ConfigXML.getInstance();
        Locale.setDefault(new Locale(c.getLocale()));

        MainGUI a = new MainGUI("seller3@gmail.com");
        a.setVisible(true);

        try {

            BLFacade appFacadeInterface;
            UIManager.setLookAndFeel("javax.swing.plaf.metal.MetalLookAndFeel");

            boolean isLocal = c.isBusinessLogicLocal();
            BLFactory factory = new BLFactory();
            appFacadeInterface = factory.getBusinessLogicFactory(isLocal);

            MainGUI.setBussinessLogic(appFacadeInterface);
            // ErreklamatuGUI g=new ErreklamatuGUI();
            // g.setVisible(true);

        } catch (Exception e) {
            a.jLabelSelectOption.setText("Error: " + e.toString());
            a.jLabelSelectOption.setForeground(Color.RED);

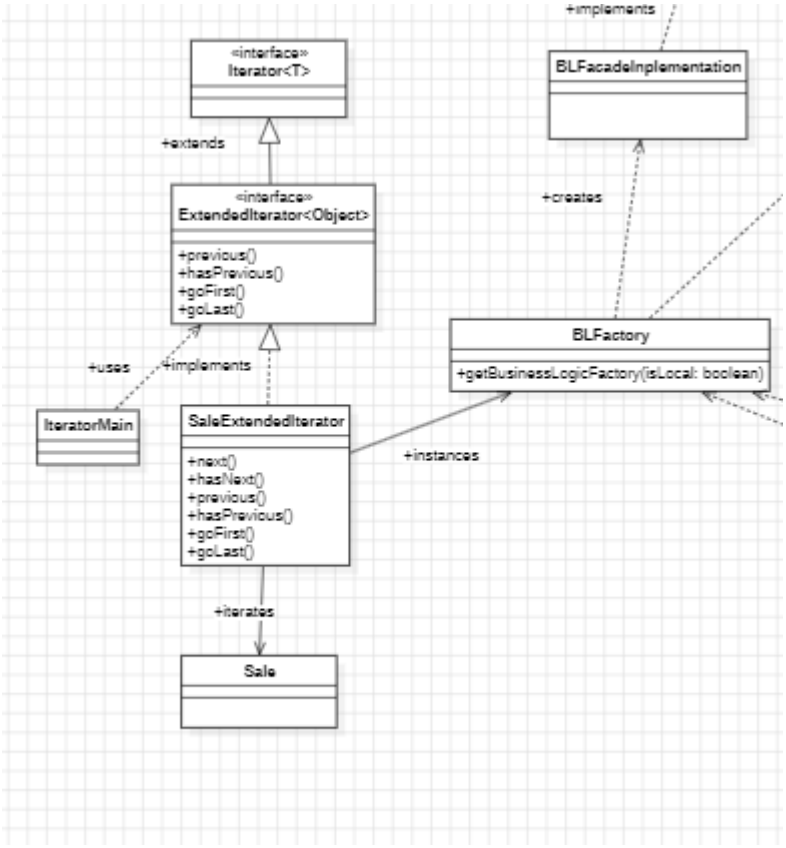
            System.out.println("Error in ApplicationLauncher: " + e.toString());
        }

    }

}
```

2. Iterator Patroia

2.1 Lortutako UMLa



Irudian ikusten dugun UMLan ikusten den bezala, ExtendIterator bat dugu Iterator interfazetik datorrena. Interfaze honek lau metodo berri ditu, previous,hasprevious,goFirst eta goLast. Beraz, iterator honekin guk nahi dugun soluzioa lortu dezakegu atzetik aurrera joateko gaitasuna baitu orain,. Azkein,SaleExtendedIterator klasea sortu da. Honek gure emaitza lortzeko beharrezkoak diren metodo guztiak ditu inplementatuta eta horrela Sale-ez osatutako iterator bat dugu atzetik aurrera eta aurretik atzera ibilitzeko gaitasunarekin.

2.2 Egindako kode aldaketak

Egin den kode aldaketa nagusienetakoa SaleExtendedIterator klasearen inplementazioa izan da.Klase honek iterator batek beharrezkoak dituen metodo guztiak ditu inplementatuta. Lehen metodoa hasNext da, metodo honek uneko posizioa alderatzen du salen listaren luzerarekin. Bigarren metodoa next da, honek berriz uneko posizioa gehi bat egin eta posizio horretan dagoen sale itzutzen du. Hirugarrena hasPrevious da, metodo honek uneko posizioa zero baino altuagoa den itzultzen du. Laugarren previous da eta uneko posizioari bat kenduz dagoen sale itzultzen du. Amaitzeko goFirts eta goLast metodoak daude. Lehenak uneko posizioari 0 balioa ematen dio. goLastek berriz uneko posizioari listaren luzera ken bat balioa ematen dio. Honako kodea gelditu da klase honetan:

```java
public class SaleExtendedIterator implements ExtendedIterator<Sale> {
    private List<Sale> sales;
    private int position;

    public SaleExtendedIterator(List<Sale> sales) {
        this.sales = sales;
        this.position = 0;
    }

    @Override
    public boolean hasNext() {
        return position < sales.size();
    }

    @Override
    public Sale next() {
        return sales.get(position++);
    }

    @Override
    public boolean hasPrevious() {
        return position >= 0;
    }

    @Override
    public Sale previous() {
        return sales.get(position--);
    }

    @Override
    public void goFirst() {
        position = 0;
    }

    @Override
    public void goLast() {
        if (sales != null && !sales.isEmpty()) {
            position = sales.size() - 1;
        } else {
            position = 0;
        }
    }
```

Iterator bat sortzeko datubaseko hainbata eragiketa egin behar dira, beraz BLFacade eta BLFacadeImplementation klaseetan metodo berri bat gehitu da iterator bat sortu ahal izateko. Honako metodoa sortu da BLFacadeImplementation klasean:

```java
@Override
public ExtendedIterator<Sale> getSalesIterator(String desc) {
    dbManager.open();
    List<Sale> sales=dbManager.getSales(desc);
    dbManager.close();
    ExtendedIterator<Sale> saleiterator= new SaleExtendedIterator(sales);
    return saleiterator;
}
```

Ondoren IteratorMain klasean metodo hau baliatuko da iterator bat sortzerako orduan.

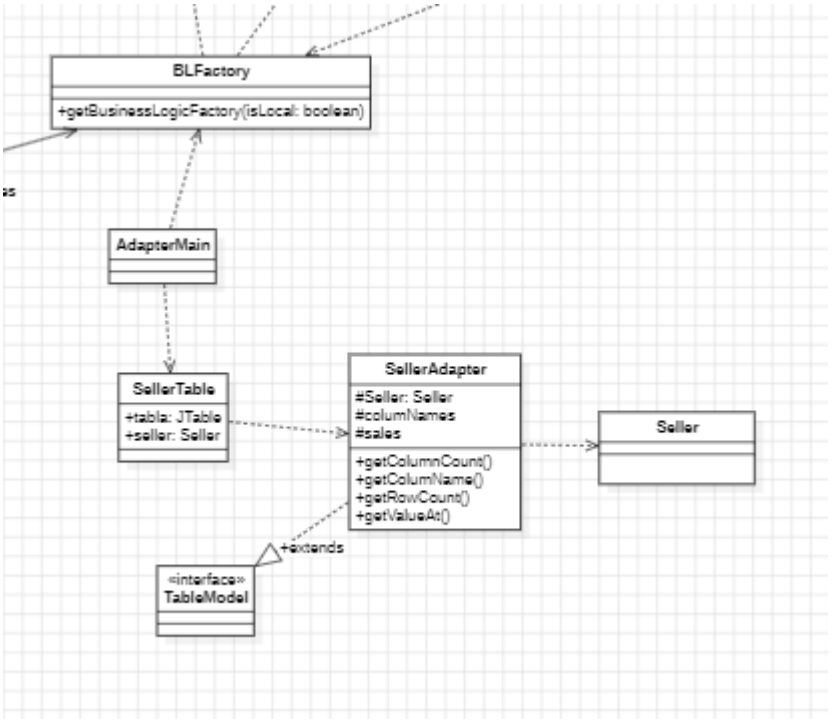
2.3 Lortutako emaitzak

```
Read from config.xml:    businessLogicLocal=true        databaseLocal=true      dataBaseInitialized=true
oct 29, 2026 3:19:10 P. M. dataAccess.DataAccess <init>
INFORMACIÓN: File deleted
oct 29, 2026 3:19:11 P. M. dataAccess.DataAccess open
INFORMACIÓN: DataAccess opened => isDatabaseLocal: true
oct 29, 2026 3:19:11 P. M. dataAccess.DataAccess initializeDB
INFORMACIÓN: Db initialized
DataAccess created => isDatabaseLocal: true isDatabaseInitialized: true
DataAcess closed
oct 29, 2026 3:19:11 P. M. businesslogic.BLFacadeImplementation <init>
INFORMACIÓN: Creating BLFacadeImplementation instance with DataAccess parameter
oct 29, 2026 3:19:11 P. M. dataAccess.DataAccess open
INFORMACIÓN: DataAccess opened => isDatabaseLocal: true
>> DataAccess: getProducts=> from=
DataAcess closed
_____________________
FROM    LAST    TO      FIRST
8;sukaldeko mahaia;45.0
7;polar kilor erlojua;30.0
6;orbea mendiko bizikleta;225.0
5;iphone 17;400.0
4;imac 27;200.0
3;samsung 42" telebista;175.0
2;salomon mendiko botak;20.0
1;futbol baloia;10.0

_____________________
FROM FIRST TO LAST
1;futbol baloia;10.0
2;salomon mendiko botak;20.0
3;samsung 42" telebista;175.0
4;imac 27;200.0
5;iphone 17;400.0
6;orbea mendiko bizikleta;225.0
7;polar kilor erlojua;30.0
8;sukaldeko mahaia;45.0
```

3. Adapter Patroia

3.1 Lortutako UML



Honako UMLa garatu da. UML honi esker orain SellerAdapter izeneko klaseak gure azken emaitza lortzeko beharrezko inplementazioa du, lehenagotik sortuta zegoen TableModel-en inplementazioa jarraituz. Taula hori lortzeko SellerTable klase bat dago eta honek SellerAdapter instantzia bat sortuko du, eta ondoren, Jtable bat sortuko du parametro gisa sortu dugun SellerAdapter pasatuz. Horri esker, taula berria sortuko da. Proba hauek ikuskatzeko AdapterMain izeneko klase bat erabili da. BLFactory erabiliz gure negozio logika sortuko da eta bertako metodoekin gure emaitza lortu ahal izango dugu.

3.2 Egindako kode aldaketak

Egin den kode aldaketa nagusia izan da SellerAdapter klasearen sorrera. Klase honek AbstractTableModel-en extends izan behar du horrela ondoren JTable bat sortzerakoan adapterra parametro gisa pasa dadin. Hiru atributu izango ditu, batetik, seller bat izango du ondoren bere datuak jaso ahal izateko. Bigarrenik, gure taulak izango dituen zutabeen izenak gordeko dira. Azkenik, bektore bat izango dugu bertan Seller dituen sale guztiak jasotzeko. Lau metodo ezberdin inplementatu dira. Lehen metodoak taulak dituen zutabe kopurua itzultzen du. Bigarrenak berriz, parametro gisa zenbaki bat jasota, zenbaki horretan dagoen zutabearen izena itzultzen du. Hirugarren metodoak taulak dituen lerro kopurua itzultzen du, horretarako Seller-ak dituen sale kopurua erabiltzen du. Laugarren eta azken metodoak berriz, ilara eta zutabe zenbaki bat jasotzen ditu eta balio horietan kokatutako elementua itzultzen du. Horretarako, zutabearen zenbakia begiratzen du, balioa 0 bada, gure sales bektoretik hartzen du dagokiona eta bere izenburua itzultzen du. Balioa 1 denean dagokion salearen deskripzioa itzultzen du. Balio 3 den kasuan berriz salearen prezioa itzuliko du. Honako kodea gelditzuko litzateke SellerAdapter klasean:

```java
public class SellerAdapter extends AbstractTableModel {
    protected Seller seller;
    protected String[] columnNames = new String[] { "title", "description", "price" };
    protected Vector<Sale> sales;

    public SellerAdapter(Seller s) {
        this.seller = s;
        sales = new Vector<Sale>(s.getSales());
    }

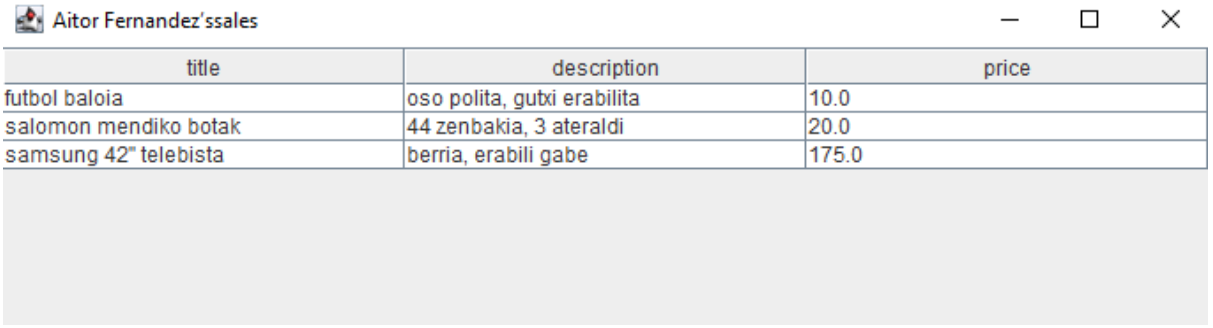
    public int getColumnCount() {
        // Challenge!
        return columnNames.length;
    }

    public String getColumnName(int i) {
        // Challenge!
        return columnNames[i];
    }

    public int getRowCount() {
        // Challenge!
        return seller.getSales().size();
    }

    public Object getValueAt(int row, int col) {
        // Challenge!
        if (col == 0) {
            return sales.get(row).getTitle();
        } else if (col == 1) {
            return sales.get(row).getDescription();
        } else {
            return sales.get(row).getPrice();
        }
    }
}
```

3.3 Lortutako emaitzak



title	description	price
futbol baloia	oso polita, gutxi erabilita	10.0
salomon mendiko botak	44 zenbakia, 3 ateraldi	20.0
samsung 42" telebista	berria, erabili gabe	175.0